

Ingress Controller

Class no:3

Ingress

In Kubernetes, **Ingress** is an API object that manages external access to the services within a cluster, typically HTTP and HTTPS.

Think of it as a **smart traffic controller** or a **reverse proxy/load balancer** that sits at the edge of your cluster. It provides routing rules to direct incoming internet traffic to the correct internal microservices without you having to expose each service individually.

Why Do You Need It? (The Problem it Solves)

Without Ingress, if you want to expose a service to the outside world, you usually have to use a service type like NodePort or LoadBalancer.

- **NodePort** opens a specific port on all your nodes (e.g., 32000), which is messy and insecure.
- **LoadBalancer** creates a cloud provider load balancer (like AWS ALB or Google Cloud NLB) for *every single service*. If you have 20 microservices, that means 20 expensive cloud load balancers.

Ingress solves this by acting as a single entry point. You only need *one* cloud load balancer pointing to your Ingress, and Ingress handles the internal routing based on the request's URL or path.

Key Features of Ingress

- **Content-Based Routing:** Route traffic based on the HTTP request path (e.g., example.com/api goes to Service A, while example.com/shop goes to Service B).
- **Host-Based Routing:** Route traffic based on the domain name (e.g., api.example.com goes to one service, app.example.com goes to another).
- **SSL/TLS Termination:** Secure your applications easily by handling SSL certificates at the Ingress level, so your internal services don't have to manage them.
- **Name-Based Virtual Hosting:** Support multiple domain names using the same single IP address.

The Two Core Components

To make Ingress work, you actually need two different things in your cluster:

1. **The Ingress Object (The Rulebook):** This is a YAML file where you define your routing rules. It tells Kubernetes *where* traffic should go.
2. **The Ingress Controller (The Engine):** The Ingress object does nothing on its own. You must install an Ingress Controller (like **NGINX Ingress Controller**, **Traefik**, **Emissary-ingress**, or **HAProxy**). The controller constantly monitors the cluster for Ingress objects and actually carries out the routing and proxying.

Hands on: Ingress controller setup and exposing the contents using the Load balancer DNS with it.

Step no:1 Create a cluster called: **mycluster** with **two nodes**. To access them create a instance called: **EKS**

Configure it via the access keys and secreta access keys.

Then connect the aws with the eks: **aws eks --region ap-south-1 update-kubeconfig --name mycluster**

Install Kubectl

```
[ec2-user@ip-172-31-8-104 ~]$ aws eks --region ap-south-1 update-kubeconfig --name mycluster
Added new context arn:aws:eks:ap-south-1:314383175192:cluster/mycluster to /home/ec2-user/.kube/config
[ec2-user@ip-172-31-8-104 ~]$ curl -Lo kubectl https://dl.k8s.io/release/$(curl -s -L https://dl.k8s.io/release/latest-stable-gcs.yaml)
chmod +x ./kubectl
sudo mv ./kubectl /usr/local/bin/kubectl
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 58160k 100 58160k    0     0  368.2M      0  --:--:-- --:--:-- --:--:-- 368.8M
```

Step no:2 Install eksctl

```
[ec2-user@ip-172-31-8-104 ~]$ # Install eksctl
ARCH=amd64
PLATFORM=$(uname -s)_$ARCH

curl -sLO "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_$(uname -s)_$(uname -m).tar.gz"
curl -sL "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_$(uname -s)_$(uname -m).tar.gz"

tar -xzf eksctl_$(uname -s)_$(uname -m).tar.gz -C /tmp && rm eksctl_$(uname -s)_$(uname -m).tar.gz
sudo install -m 0755 /tmp/eksctl /usr/local/bin && rm /tmp/eksctl
eksctl Linux amd64.tar.gz: OK
```

OIDC and IAM Setup

```
[ec2-user@ip-172-31-8-104 ~]$ eksctl utils associate-iam-oidc-provider \
  --region=ap-south-1 \
  --cluster=mycluster \
  --approve
2026-06-14 09:58:42 [i] will create IAM Open ID Connect provider for cluster "mycluster"
2026-06-14 09:58:42 [✓] created IAM Open ID Connect provider for cluster "mycluster"
```

Step no:3 Now we have to create an IAM user. First create the policy to attach with it.

In order to attach with the IAM user we need an IAM policy

IAM → Policies → create policy → Specify permissions: **JSON**

Copy and paste the contents in the github file

Name the policy as: **AWSLoadBalancerControllerIAMPolicy**

Create it

Step 1
Specify permissions

Step 2
Review and create

Specify permissions [Info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

Visual

JSON

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

```
    "elasticloadbalancing:CreateAction": [
      "CreateTargetGroup",
      "CreateLoadBalancer"
    ],
    "Null": {
      "aws:RequestTag/elbv2.k8s.aws/cluster": "false"
    }
  },
  {
    "Effect": "Allow",
    "Action": [
      "elasticloadbalancing:RegisterTargets",
      "elasticloadbalancing:DeregisterTargets"
    ],
    "Resource": "arn:aws:elasticloadbalancing:*:*:targetgroup/*/*"
  },
  {
    "Effect": "Allow",
    "Action": [
      "elasticloadbalancing:SetWebAcl",
      "elasticloadbalancing:ModifyListener",
      "elasticloadbalancing:AddListenerCertificates",
      "elasticloadbalancing:RemoveListenerCertificates",
      "elasticloadbalancing:ModifyRule",
      "elasticloadbalancing:SetRulePriorities"
    ],
    "Resource": "*"
  }
]
```

Edit statement

Select an exit point

+

251:2 JSON

Policy details

Policy name

Enter a meaningful name to identify this policy.

AWSLoadBalancerControllerIAMPolicy

Maximum 128 characters. Use alphanumeric and '+,=, @, _' characters.

Description - optional

Add a short explanation for this policy.

Maximum 1,000 characters. Use alphanumeric and '+,=, @, _' characters.

Permissions defined in this policy [Info](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), you must specify the permissions in the policy document.

Q Search

Allow (9 of 471 services)

Service	Access level	Resource	Request condition
Certificate Manager	Limited: List, Read	All resources	None
Cognito User Pools	Limited: Read	All resources	None
EC2	Full: Tagging Limited: List, Read, Write	Multiple	Multiple
ELB	Full: List, Tagging Limited: Read, Write	Multiple	Multiple
ELB v2	Full: Tagging Limited: List, Read, Write	Multiple	Multiple
IAM	Limited: List, Read, Write	All resources	iam:AWSServiceNan

AWSLoadBalancerControllerIAMPolicy [Info](#)

[Edit](#)[Delete](#)

Policy details

Type Customer managed	Creation time June 14, 2026, 15:37 (UTC+05:30)	Edited time June 14, 2026, 15:37 (UTC+05:30)	ARN  arn:aws:iam::314383175192:policy/AWSLoadBalancerControllerIAMPolicy
--------------------------	---	---	--

[Permissions](#)[Entities attached](#)[Tags](#)[Policy versions \(1\)](#)[Last Accessed](#)

Permissions defined in this policy [Info](#)

[Edit](#)[Summary](#)[JSON](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it.

Now create the **IAM user with respective region and arn of the policy** we have created. A service account will be created.

```
[ec2-user@ip-172-31-8-104 ~]$ eksctl create iamserviceaccount \  
--cluster mycluster \  
--region ap-south-1 \  
--namespace kube-system \  
--name aws-load-balancer-controller2 \  
--attach-policy-arn arn:aws:iam::314383175192:policy/AWSLoadBalancerControllerIAMPolicy \  
--approve
```

To see the service account info:

```
kubectl get sa aws-load-balancer-controller2 \  
-n kube-system -o yaml
```

```
[ec2-user@ip-172-31-8-104 ~]$ kubectl get sa aws-load-balancer-controller2 \  
-n kube-system -o yaml  
apiVersion: v1  
kind: ServiceAccount  
metadata:  
  annotations:  
    eks.amazonaws.com/role-arn: arn:aws:iam::314383175192:role/eksctl-mycluster-addo  
  creationTimestamp: "2026-06-14T10:09:59Z"  
  labels:  
    app.kubernetes.io/managed-by: eksctl  
  name: aws-load-balancer-controller2  
  namespace: kube-system  
  resourceVersion: "3536"  
  uid: 316ab4be-33c8-4786-95a5-53df38668c4d
```

To see the service account running in the kube-system namespace:

```
kubectl get sa -n kube-system
```

```
[ec2-user@ip-172-31-8-104 ~]$ kubectl get sa -n kube-system  
NAME                                AGE  
attachdetach-controller            25m  
aws-cloud-provider                  26m  
aws-load-balancer-controller2       7m13s
```

Step no:4 Now we have to **install and setup helm**. It is the **package manager for Kubernetes**.

```
[ec2-user@ip-172-31-8-104 ~]$ curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/master/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
[WARNING] Could not find git. It is required for plugin installation.
Downloading https://get.helm.sh/helm-v3.21.1-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
```

Add and update the repo of helm.

```
[ec2-user@ip-172-31-8-104 ~]$ helm repo add eks https://aws.github.io/eks-charts
helm repo update
"eks" has been added to your repositories
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "eks" chart repository
Update Complete. *Happy Helming!*
```

Enter the following command in the CLI to get our **VPC id**.

```
[ec2-user@ip-172-31-8-104 ~]$ aws eks describe-cluster \
  --name mycluster \
  --region ap-south-1 \
  --query "cluster.resourcesVpcConfig.vpcId" \
  --output text
vpc-0538c231a226fe3c9
```

We can cross check it with the VPC id via console.

vpc-0538c231a226fe3c9

Details [Info](#)

VPC ID

 vpc-0538c231a226fe3c9

DNS resolution

Enabled

State

 Available

Tenancy

default

Step no:4 We can install the AWS ingress Controller now with the support of helm package manager.

In the code provide the respective cluster name, region and the VPC ID.

We can see the installation is successful.

```
[ec2-user@ip-172-31-8-104 ~]$ helm install aws-load-balancer-controll
-n kube-system \
--set clusterName=mycluster \
--set region=ap-south-1 \
--set vpcId=vpc-0538c231a226fe3c9 \
--set serviceAccount.create=false \
--set serviceAccount.name=aws-load-balancer-controller2
NAME: aws-load-balancer-controller
LAST DEPLOYED: Sun Jun 14 10:38:05 2026
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
AWS Load Balancer controller installed!
```

Now we have to create two namespaces for the execution of two deployment file.

Create a file called **namespace.yaml**

Copy the contents.

```
[ec2-user@ip-172-31-8-104 ~]$ vi namespace.yaml
[ec2-user@ip-172-31-8-104 ~]$ cat namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: namespace1
---
apiVersion: v1
kind: Namespace
metadata:
  name: namespace2
```

Execute it: **kubectl apply -f namespace.yaml**

We can see two namespaces has been created.

We can also create a namespace using: **kubectl create namespace nithya**

```
[ec2-user@ip-172-31-8-104 ~]$ kubectl apply -f namespace.yaml
namespace/namespace1 created
namespace/namespace2 created
[ec2-user@ip-172-31-8-104 ~]$ kubectl create namespace nithya
namespace/nithya created
```

Once the namespace has been created, now create the deployment file for respective namespaces. We will expose them using the ingress Load Controller.

For namespace1 → app1.yaml

For namespace2 → app2.yaml

```
[ec2-user@ip-172-31-8-104 ~]$ vi app1.yaml
[ec2-user@ip-172-31-8-104 ~]$ cat app1.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app1
  namespace: namespace1
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app1
  template:
    metadata:
      labels:
        app: app1
    spec:
```

Apply the file: **kubectl apply -f app1.yaml**

Similarly, for the app2.yaml as well.

```
[ec2-user@ip-172-31-8-104 ~]$ kubectl apply -f app1.yaml
```

We can see the ingress has been created

```
[ec2-user@ip-172-31-8-104 ~]$ vi app2.yaml
[ec2-user@ip-172-31-8-104 ~]$ kubectl apply -f app2.yaml
deployment.apps/app2 created
service/namespace2-svc created
Warning: annotation "kubernetes.io/ingress.class" is deprecated,
ingress.networking.k8s.io/namespace2-ingress created
```

As the applications run inside our pods, Check they are running inside our created namespace:

kubectl get pods -n namespace1

To see the nodes: **kubectl get nodes**

```
[ec2-user@ip-172-31-8-104 ~]$ kubectl get pods -n namespace1
NAME                                READY   STATUS    RESTARTS   AGE
app1-76c6f4b5bc-dp479              1/1     Running   0           8m57s
app1-76c6f4b5bc-z42m7              1/1     Running   0           8m57s
[ec2-user@ip-172-31-8-104 ~]$ kubectl get nodes
NAME                                STATUS   ROLES    AGE
ip-172-31-27-187.ap-south-1.compute.internal  Ready   <none>   7m48s
ip-172-31-36-11.ap-south-1.compute.internal   Ready   <none>   7m48s
[ec2-user@ip-172-31-8-104 ~]$
```

Step no:5 Now a load balancer will be created for exposing it.

Click on LB → Copy the dns.

The screenshot shows the AWS Management Console interface for an Elastic Load Balancing (ELB) instance. The instance name is 'k8s-sharedalb-2bb18182fb'. The details section shows the following information:

- Load balancer type:** Application
- Status:** Active
- VPC:** vpc-0538c231a226fe3c9
- Load balancer IP address type:** IPv4
- Scheme:** Internet-facing
- Hosted zone:** ZP97RAFLXTNZK
- Availability Zones:** subnet-01bf4d440c0912db3 (aps1-az1), subnet-0c305b80f72541592 (aps1-az2), subnet-0b3c21b6de4314d95 (aps1-1b)
- Date created:** June 14, 2026, 16:24 (UTC+05:30)
- Load balancer ARN:** arn:aws:elasticloadbalancing:ap-south-1:314383175192:loadbalancer/app/k8s-sharedalb-2bb18182fb/66a63b16843d052f
- Load balancer DNS name:** k8s-sharedalb-2bb18182fb-2074409464.ap-south-1.elb.amazonaws.com (A Record)

A tooltip indicates that the DNS name has been copied.

Open a private browser paste the dns/namespace1. We can see the output of the app1.yaml file of the deployment.

The screenshot shows a web browser window with the address bar displaying 'k8s-sharedalb-2bb18182fb-2074409464.ap-south-1.elb.amazonaws.com/firstnamespace'. The page content shows 'Namespace1 result'.

Open a private browser paste the dns/namespace2. We can see the output of the app2.yaml file of the deployment

The screenshot shows a web browser window with the address bar displaying 'k8s-sharedalb-2bb18182fb-2074409464.ap-south-1.elb.amazonaws.com/secondnamespace'. The page content shows 'Namespace2 result'.

RBAC

Role-Based Access Control (RBAC) in Kubernetes is the system that regulates access to resources within your cluster based on the roles of individual users or applications.

Think of it as the digital security guard of your cluster. Instead of giving everyone keys to every room, you give people keys based on their job description.

The Four Core Components

RBAC is managed using four standard Kubernetes API objects, split into two categories: **What you can do (Roles)** and **Who can do it (Bindings)**.

1. Defining the Permissions (The "What")

These objects specify a set of rules (verbs like get, list, create, delete) that can be performed on specific resources (like pods, services, deployments).

- **Role:** Applies to a **single, specific namespace**. Use this to grant access to resources within one project or team environment.
- **ClusterRole:** Applies **cluster-wide**. Use this to grant access to non-namespaced resources (like nodes or persistentvolumes) or to grant the same permissions across all namespaces.

2. Granting the Permissions (The "Who")

These objects connect the Roles to "subjects" (Users, Groups, or ServiceAccounts).

- **RoleBinding:** Grants permissions within a **specific namespace**. It can link a Role to a user, or even link a ClusterRole to a user *scoped down* to that single namespace.
- **ClusterRoleBinding:** Grants permissions **across the entire cluster**, applicable to all namespaces.

Quick Reference Table

API Object	Scope	Best Used For...
Role	Namespace	Allowing a developer to manage pods <i>only</i> in the development namespace.
ClusterRole	Cluster-wide	Allowing an admin to monitor cluster nodes or view all secrets everywhere.
RoleBinding	Namespace	Binding a Role to a QA engineer for the testing namespace.
ClusterRoleBinding	Cluster-wide	Binding a ClusterRole to a CI/CD tool so it can deploy anywhere in the cluster.

Hands-on: Setting RBAC for users accessing Jenkins server.

Step no:1 Create two IAM users **developer** and **tester** with console access.

IAM users (0) Info

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search

User namePathGroupLast activityMFAPassword ageConsole last sign-inAccess key IDActive key ageAcc

No resources to display

IAM users (2) Info

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search

User namePath

developer/

tester/

Provide them admin access for Jenkins.

developer > Add permissions

Add permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐ Add user to group

Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions

Copy all group memberships, attached managed policies, inline policies, and any existing permissions boundaries from an existing user.

☒ Attach policies directly

Attach a managed policy directly to a user. As a best practice, attach policies to a group instead. Then, add the user to the group.

Permissions policies (1/1506)

Search

Filter by Type

All types

Policy nameTypeAttached entities

AccessAnalyzerServiceRolePolicy

AWS managed

0

AccountManagementFromVercel

AWS managed

0

☒

AdministratorAccess

AWS managed - job function

3

tester > Add permissions

Add permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐ Add user to group

Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions

Copy all group memberships, attached managed policies, inline policies, and any existing permissions boundaries from an existing user.

☒ Attach policies directly

Attach a managed policy directly to a user. As a best practice, attach policies to a group instead. Then, add the user to the group.

Permissions policies (1/1506)

Search

Filter by Type

All types

Policy nameTypeAttached entities

AccessAnalyzerServiceRolePolicy

AWS managed

0

AccountManagementFromVercel

AWS managed

0

☒

AdministratorAccess

AWS managed - job function

3

We will set the boundaries for developer and the tester inside the Jenkins server using RBAC.

Developer → one namespace with full permission

Tester → All namespace with only get and read permission

Step no:2 Check the node status

```
[ec2-user@ip-172-31-8-104 ~]$ kubectl get nodes
NAME                                     STATUS
ip-172-31-27-187.ap-south-1.compute.internal   Ready
ip-172-31-36-11.ap-south-1.compute.internal   Ready
[ec2-user@ip-172-31-8-104 ~]$
```

We need to create a policy to be attached with the respective groups which consist of the respective roles.

Now they are allowed to access EKS now, but inside EKS we are setting boundary for them using RBAC. For that open the config file: **kubectl edit configmap aws-auth -n kube-system**

paste under the map roles in eks-full-access instance maproles

mapUsers: |

- userarn: **respective arn**

username: developer

groups:

- developer-grp

- userarn: **respective arn**

username: tester

groups:

- tester-grp

```
#
apiVersion: v1
data:
  mapRoles: |
    - rolearn: arn:aws:iam::314383175192:role/AmazonEKSAutoNodeRole
      groups:
        - system:bootstrappers
        - system:nodes
      username: system:node:{{EC2PrivateDNSName}}
  mapUsers: |
    - userarn: arn:aws:iam::314383175192:user/developer
      username: developer
      groups:
        - developer-grp
    - userarn: arn:aws:iam::314383175192:user/tester
      username: tester
      groups:
        - tester-grp
kind: ConfigMap
metadata:
  creationTimestamp: "2026-06-14T10:51:27Z"
  name: aws-auth
  namespace: kube-system
  resourceVersion: "10074"
  uid: cfeb8f14-0e02-430d-b2ff-e0595e3ff16d
~
~
```

We can see the file has been edited. Ensure we are entering the respective arn.

```
[ec2-user@ip-172-31-8-104 ~]$ kubectl edit configmap aws-auth -n kube-system
configmap/aws-auth edited
```

Step no:4 We need to create namespace for role binding,

Developer namespace for developer

Tester namespace for tester.

```
Last login: Sun Jun 14 11:58:37 2026 from 13.233.177.5
[ec2-user@ip-172-31-8-104 ~]$ kubectl create namespace developer-namespace
namespace/developer-namespace created
[ec2-user@ip-172-31-8-104 ~]$ kubectl create namespace tester-namespace
namespace/tester-namespace created
```

Create a role for developer role: **vi role.yaml**

With all permissions in the developer namespace.

```
[ec2-user@ip-172-31-8-104 ~]$ vi role.yaml
[ec2-user@ip-172-31-8-104 ~]$ cat role.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: dev-role
  namespace: developer-namespace
rules:
- apiGroups: ["", "apps", "batch"]
  resources: ["pods", "services", "deployments", "jobs"]
  verbs: ["get", "list", "create", "update", "delete", "watch"]
```

Role binding for dev-role with the dev-group which has the developer. Anyone inside this group will be attached with this role.

```
[ec2-user@ip-172-31-8-104 ~]$ vi rolebinding.yaml
[ec2-user@ip-172-31-8-104 ~]$ cat rolebinding.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-binding
  namespace: developer-namespace
subjects:
- kind: Group
  name: developer-grp
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: dev-role
  apiGroup: rbac.authorization.k8s.io
```

Similar for testers.

vi clusterrole.yaml for tester roles

```
[ec2-user@ip-172-31-8-104 ~]$ vi clusterrole.yaml
[ec2-user@ip-172-31-8-104 ~]$ cat clus
cat: clus: No such file or directory
[ec2-user@ip-172-31-8-104 ~]$ cat clusterrole.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: view-role
rules:
- apiGroups: ["", "apps", "batch"]
  resources: ["pods", "services", "deployments", "jobs"]
  verbs: ["get", "list", "watch"]
```

Role binding with the tester role: **vi clusterrolebinding.yaml**

```
[ec2-user@ip-172-31-8-104 ~]$ vi clusterrolebinding.yaml
[ec2-user@ip-172-31-8-104 ~]$ cat clusterrolebinding.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: readonly-binding
subjects:
- kind: Group
  name: tester-grp
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: view-role
  apiGroup: rbac.authorization.k8s.io
```

Now apply them using: **kubectl apply -f**

```
[ec2-user@ip-172-31-8-104 ~]$ kubectl apply -f role.yaml
role.rbac.authorization.k8s.io/dev-role created
[ec2-user@ip-172-31-8-104 ~]$ kubectl apply -f rolebinding.yaml
rolebinding.rbac.authorization.k8s.io/dev-binding created
[ec2-user@ip-172-31-8-104 ~]$ kubectl apply -f clusterrole.yaml
clusterrole.rbac.authorization.k8s.io/view-role created
[ec2-user@ip-172-31-8-104 ~]$ kubectl apply -f clusterrolebinding.yaml
clusterrolebinding.rbac.authorization.k8s.io/readonly-binding created
```

Step no:5 Now we have to create two instances. Because our current EKS instance has root level access.

So for this role create respective instances.

1. Developer-instance
2. Tester-instance

☐	Name	Instance ID	Instance state	Instance type
☐		i-01efbf95a349a3989	Running	c7i-flex.large
☐		i-04c06b5d9a8f7bd7d	Running	c7i-flex.large
☐	EKS	i-0ad9e860db54562a0	Running	t3.small
☐	tester-instance	i-0f287167fd8717822	Running	t3.micro
☐	developer-instance	i-0c8c27a726d27146e	Running	t3.micro

Connect both instances.

Now we need to provide access key & secret access key for the respective instances.

Go to **IAM user** → **developer IAM user** → security credentials → create access key → tag it as: **dev** → create access key → copy the access key.

developer Info

[Delete](#)

Summary

ARN
[arn:aws:iam::314383175192:user/developer](#)

Created
June 14, 2026, 17:24 (UTC+05:30)

Console access
 Enabled without MFA

Last console sign-in
 Never

Access key 1
[Create access key](#)

Permissions Groups Tags **Security credentials** Last Accessed

Console sign-in

[Manage console access](#)

Console sign-in link
<https://314383175192.signin.aws.amazon.com/console>

Console password
Updated 36 minutes ago (2026-06-14 17:24 GMT+5:30)

Last console sign-in
 Never

Multi-factor authentication (MFA) (0)

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. [Learn more](#)

[Remove](#)
[Resync](#)
[Assign MFA device](#)

Type

Identifier

Certifications

Created on

No MFA devices. Assign an MFA device to improve the security of your AWS environment

[Assign MFA device](#)

Access keys (0)

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

[Create access key](#)

No access keys. As a best practice, avoid using long-term credentials like access keys. Instead, use tools which provide short-term credentials. [Learn more](#)

Access key best practices & alternatives Info

Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.

Use case

☒ **Command Line Interface (CLI)**
You plan to use this access key to enable the AWS CLI to access your AWS account.

☐ **Local code**
You plan to use this access key to enable application code in a local development environment to access your AWS account.

☐ **Application running on an AWS compute service**
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

☐ **Third-party service**
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.

☐ **Application running outside AWS**
You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.

☐ **Other**
Your use case is not listed here.

Alternatives recommended

- Use AWS CLI V2 and the `aws login` command to use your existing console credentials in the CLI. [Learn more](#)
- Use AWS CloudShell, a browser-based CLI, to run commands. [Learn more](#)

Confirmation

☒ I understand the above recommendation and want to proceed to create an access key.

[Cancel](#)
[Next](#)

Set description tag - optional [Info](#)

The description for this access key will be attached to this user as a tag and shown alongside the access key.

Description tag value

Describe the purpose of this access key and where it will be used. A good description will help you rotate this access key confidently later.

dev

Maximum 256 characters. Allowed characters are letters, numbers, spaces representable in UTF-8, and: _ . : / = + - @

Cancel

Previous

Create access key

```
/mnt/
[ec2-user@ip-172-31-14-168 ~]$ aws configure
AWS Access Key ID [None]: AKIAUSMVSRFMLGKGA5XC
AWS Secret Access Key [None]: J0feSPpB+d/JUh69WIIjX
Default region name [None]: Default output format [None]:
[ec2-user@ip-172-31-14-168 ~]$
```

i-0c8c27a726d27146e (developer-instance)

PublicIPs: 65.0.27.47 PrivateIPs: 172.31.14.168

Similarly for tester as well.

tester [Info](#) [Delete](#)

Summary

ARN

arn:aws:iam::314383175192:user/tester

Created

June 14, 2026, 17:25 (UTC+05:30)

Console access

Enabled without MFA

Last console sign-in

Never

Access key 1

Create access key

Permissions

Groups

Tags

Security credentials

Last Accessed

Console sign-in

Console sign-in link

https://314383175192.signin.aws.amazon.com/console

Console password

Updated 38 minutes ago (2026-06-14 17:25 GMT+5:30)

Last console sign-in

Never

Manage console access

Multi-factor authentication (MFA) (0)

Remove

Resync

Assign MFA device

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. [Learn more](#)

Type	Identifier	Certifications	Created on
No MFA devices. Assign an MFA device to improve the security of your AWS environment			

Assign MFA device

Access keys (0)

Create access key

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

Access key best practices & alternatives [Info](#)

Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.

Use case

☒ **Command Line Interface (CLI)**

You plan to use this access key to enable the AWS CLI to access your AWS account.

☐ **Local code**

You plan to use this access key to enable application code in a local development environment to access your AWS account.

☐ **Application running on an AWS compute service**

You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

☐ **Third-party service**

You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.

☐ **Application running outside AWS**

You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.

☐ **Other**

Your use case is not listed here.

⚠ Alternatives recommended

- Use AWS CLI V2 and the `aws login` command to use your existing console credentials in the CLI. [Learn more](#)
- Use AWS CloudShell, a browser-based CLI, to run commands. [Learn more](#)

Confirmation

☒ I understand the above recommendation and want to proceed to create an access key.

[Cancel](#)

[Next](#)

Set description tag - optional [Info](#)

The description for this access key will be attached to this user as a tag and shown alongside the access key.

Description tag value

Describe the purpose of this access key and where it will be used. A good description will help you rotate this access key confidently later.

test

Maximum 256 characters. Allowed characters are letters, numbers, spaces representable in UTF-8, and: _ . : / = + - @

[Cancel](#)

[Previous](#)

[Create access key](#)

```
[ec2-user@ip-172-31-4-225 ~]$ aws configure
AWS Access Key ID [None]: AKIAUSMVSRYMD3XV4HOV
AWS Secret Access Key [None]: RLbRNUBl0l1n3r3jrN+
Default region name [None]: Default output format
[ec2-user@ip-172-31-4-225 ~]$
```

i-0f287167fd8717822 (tester-instance)

PublicIPs: 13.201.99.183 PrivateIPs: 172.31.4.225

Step no:6 Install Kubernetes in both instances.

```
[ec2-user@ip-172-31-14-168 ~]$ aws eks update-kubeconfig --region ap-south-1 --name mycluster
Added new context arn:aws:eks:ap-south-1:314383175192:cluster/mycluster to /home/ec2-user/.kube
[ec2-user@ip-172-31-14-168 ~]$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/
chmod +x kubectl
sudo mv kubectl /usr/local/bin/

kubectl version
 % Total      % Received % Xferd  Average Speed   Time    Time     Time  Current
             Dload  Upload   Total     Spent    Left     Speed
100 58160k 100 58160k    0     0 384.4M      0  --:--:-- --:--:-- --:--:-- 386.3M
Client Version: v1.36.2
Kustomize Version: v5.8.1
Server Version: v1.35.5-eks-0247562
[ec2-user@ip-172-31-14-168 ~]$
```

i-0c8c27a726d27146e (developer-instance)

PublicIPs: 65.0.27.47 PrivateIPs: 172.31.14.168

```
[ec2-user@ip-172-31-4-225 ~]$ aws eks update-kubeconfig --region ap-south-1 --name mycluster
Added new context arn:aws:eks:ap-south-1:314383175192:cluster/mycluster to /home/ec2-user/.ku
[ec2-user@ip-172-31-4-225 ~]$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s
chmod +x kubectl
sudo mv kubectl /usr/local/bin/

kubectl version
 % Total      % Received % Xferd  Average Speed   Time    Time     Time  Current
             Dload  Upload   Total     Spent    Left     Speed
100 58160k 100 58160k    0     0 349.3M      0  --:--:-- --:--:-- --:--:-- 348.4M
Client Version: v1.36.2
Kustomize Version: v5.8.1
Server Version: v1.35.5-eks-0247562
[ec2-user@ip-172-31-4-225 ~]$
```

i-0f287167fd8717822 (tester-instance)

PublicIPs: 13.201.99.183 PrivateIPs: 172.31.4.225

Connect and get inside of the developer -instance.

Create a pod inside it using: **vi pods.yaml**

```
[ec2-user@ip-172-31-14-168 ~]$ vi pods.yaml
[ec2-user@ip-172-31-14-168 ~]$ cat pods.yaml
apiVersion: v1
kind: Pod
metadata:
  name: dev-pod
  namespace: developer-namespace
  labels:
    app: dev-app
spec:
  containers:
    - name: nginx-container
      image: nginx:latest
      ports:
        - containerPort: 80
```

Apply the pods.

Now open the developer namespace and check we can see the pods.

Try to open other namespace, it will show forbidden.

```
ContainerError: 80
[ec2-user@ip-172-31-14-168 ~]$ kubectl apply -f pods.yaml
pod/dev-pod created
[ec2-user@ip-172-31-14-168 ~]$ kubectl get pods -n developer-namespace
NAME          READY   STATUS    RESTARTS   AGE
dev-pod       1/1     Running   0           2m55s
[ec2-user@ip-172-31-14-168 ~]$
```

i-0c8c27a726d27146e (developer-instance)

PublicIPs: 65.0.27.47 PrivateIPs: 172.31.14.168

In tester instance → get all pods info. We can see all the info about all pods.

```
[ec2-user@ip-172-31-4-225 ~]$ kubectl get pods -A
NAMESPACE          NAME
developer-namespace dev-pod
external-dns        external-dns-6855cd5fc-545rv
kube-system         aws-load-balancer-controller-749b6db4c6-
kube-system         aws-load-balancer-controller-749b6db4c6-
kube-system         aws-node-hwhx4
kube-system         aws-node-tmnw6
kube-system         coredns-5c944bb4f4-jlvdj
kube-system         coredns-5c944bb4f4-zpngt
kube-system         ebs-csi-controller-7879885cf-2pt5q
kube-system         ebs-csi-controller-7879885cf-4nxkg
kube-system         ebs-csi-node-52vdw
kube-system         ebs-csi-node-rpxzr
kube-system         eks-node-monitoring-agent-jz2q7
kube-system         eks-node-monitoring-agent-wthss
kube-system         eks-pod-identity-agent-28zz8
kube-system         eks-pod-identity-agent-hbd45
```

i-0f287167fd8717822 (tester-instance)

PublicIPs: 13.201.99.183 PrivateIPs: 172.31.4.225

Try to create a namespace inside it. It will show it is forbidden.

```
[ec2-user@ip-172-31-4-225 ~]$ kubectl create namespace nithi
Error from server (Forbidden): namespaces is forbidden: User "t
[ec2-user@ip-172-31-4-225 ~]$
```

i-0f287167fd8717822 (tester-instance)

PublicIPs: 13.201.99.183 PrivateIPs: 172.31.4.225